

See discussions, stats, and author profiles for this publication at: <https://www.researchgate.net/publication/352903131>

Gesture-Controlled Robotic Arm Utilizing OpenCV

Conference Paper in European Journal of Science and Technology · June 2021

DOI: 10.1109/HORAS52670.2021.9461389

CITATIONS

10

READS

2,361

2 authors:



Jedidiah Paterson

University of Nottingham

1 PUBLICATION 10 CITATIONS

SEE PROFILE



Ahmed Aldabbagh

University of Nottingham

11 PUBLICATIONS 30 CITATIONS

SEE PROFILE

Gesture-Controlled Robotic Arm Utilizing OpenCV

Jedidiah Paterson
Department of Electrical and Electronic Engineering
The University of Nottingham
United Kingdom
eeyjqp@nottingham.ac.uk

Dr. Ahmed Aldabbagh
Department of Electrical and Electronics Engineering
The University of Nottingham
United Kingdom
ahmed.aldabbagh@nottingham.ac.uk

Abstract— In this paper, a low-cost, 3D printed robotic arm that uses a system of gesture recognition controlled via computer vision is presented. The developed approach of Human-Computer Interaction (HCI) employs a single commonly available USB2.0 High-Definition camera to capture hand movements and gestures, implementing the OpenCV library. This enables the translation of those movements and gestures into commands that can be transferred using internet protocols to a Raspberry Pi; then interfaces with the robotic arm. Using computer vision as a method of HCI reduces components and overall cost. The Raspberry Pi uses individual commands to control the robotic arm's servo motors while monitoring multiple pressure sensors and limit switches to ensure the robotic arm operates within a certain limit of load. This prevents the arm from overextending itself while protecting the integrity of items within its grasp.

Keywords— *Computer Vision, Raspberry Pi, OpenCV, Gesture Recognition, 3D Printing*

I. INTRODUCTION

As a method of Human-Computer Interaction (HCI), hand gesture recognition can serve as a substitute to standard remote controls or keystrokes currently prescribed. It can do away with the need to learn complex control systems as it provides a natural intuitive system [1]. Presently, there are two main types of HCIs that can interpret human hand gestures. The first is the *Data Glove* method, which has the user wear a glove of some description that requires the use of accelerometers, gyroscopes, power source for wireless gloves or a network of data and power lines for a wired glove. The second method is by using a form of *Computer Vision* to isolate the hand and track its movements; in some experiments a colored glove is used to aid the tracking process. Both methods of hand gesture recognition have their arguments for and against.

With the *Data Glove* method, experiments have found that the trending accuracy tends to be higher than some computer vision methods, upwards of 95% gesture recognition due to their higher sampling rate of 100 samples per second. However, there are a few drawbacks. First is the high cost of a data glove [2-6]. Second is accelerometers drift, resulting in a continuous lowering of accuracy over extended periods of activity [2,3]. Finally, data gloves tend to be cumbersome and uncomfortable when used or long periods at a time and not robust enough for outdoor use [7].

Methods involving *Computer Vision* (CV) can range in complexity and accuracy based upon the camera arrangement that is used. A review of patents filed up to 2017 shows that many companies are researching hand gesture recognition and have patented 943 methods containing Infrared, Stereoscopic, Depth, or Time of Flight cameras [8]. The advantages of using CV as an HCI are:

- A CV system can monitor multiple hands

University of Nottingham/ Electrical and Electronic Department

- Public device can be made touchless
- Cameras are portable, often found on every cell phone
- Cameras do not impede hand movements allowing for more natural expression of hand gestures

When trying to detect hand gestures, cameras face challenges like background interference, image capture rate for real time processing, and 2-dimensional image representing of a 3-dimensional hand.

A. Background Interference

The CV method can overcome this issue by using image processing software such as OpenCV. Background illumination and noise can be edited from the picture frame using the technique of background subtraction which can remove stationary static objects from the image [9].

B. Sampling Rate

Sampling rate can be improved upon by utilizing a camera with higher specifications. While the camera in this paper only has a frame rate of 17 Frames Per Second (FPS) DesignSpark makes a high-resolution camera for the Raspberry Pi that has a 30FPS for only about 100 Great British Pounds (GBP). Further increases in frame rate can be made with more expensive cameras such as the Sony PXW-FS5 which can capture 960FPS for a limited duration, but this camera will not make the system low cost [10,11].

C. 3-Dimensional representation

Knowing where unobserved parts of the hand are located, is paramount to build a control system. Many CV applications use the Hidden Markov method to estimate the current hand gesture, based on the previous hand gestures and established Hidden Markov chains [12]. A Hidden Markov model can estimate locations of the unseen hand parts based upon their previously observed locations.

II. THE SYSTEM

A. Cameras

Two cameras were tested in this experiment. Each camera was chosen for accessibility and cost.

The first camera tested was a built-in camera on the ASUS Notebook PC. The camera is the USB2.0 HD UVC WebCam with 17FPS and a max resolution of 1280 x 720 pixels [13]. This camera was primarily chosen to see if standard built-in cameras are equipped enough to accurately capture and evaluate gestures for use in control systems.

The second camera tested was the HUE HD Camera which used a USB based plug and play concept. The HUE camera is capable of up to 30FPS at lower resolutions, but during this project we set the resolution at 1280 x 720 pixels and the camera-maintained 15–20FPS. The HUE HD camera has a

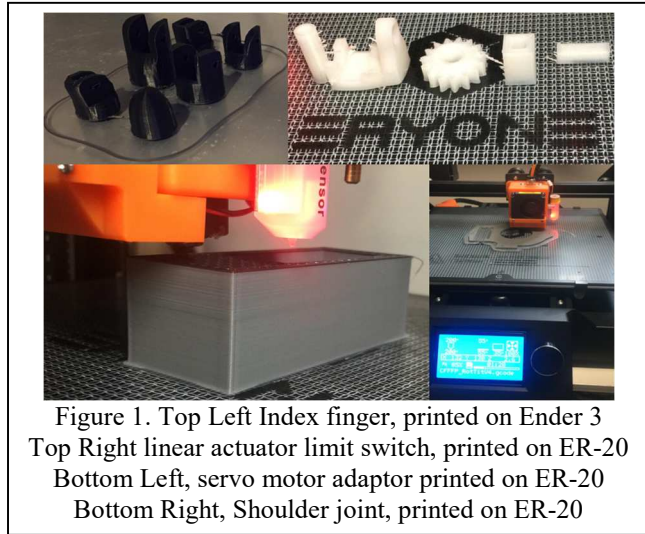
cost, 33GBP [14]. These cameras were chosen because of their low cost and similar frame rate. This allowed for the built-in camera to be tested against an external camera to evaluate its performance more accurately.

B. Raspberry Pi

A Raspberry Pi 3 Model B was used to control the robot arm. We chose the Raspberry Pi as the control mechanism because it has variety of upgradable options, 30 general-purpose input/output (GPIO) pins, software writable pulse width modulation (PWM) control, local area network accessibility.

C. 3D Printers

Two 3D printers, the Ender 3 and the EryOne ER-20, were used for printing of the robotic arm using Polylactic Acid (PLA) filament.



D. Robotic Arm

The robotic arm was based on the InMoov robot designed by Gaël Langevin [15]. The build of the arm contains several servo motors. Two TowerPro MG946R motors were used for the Shoulder and Bicep joints. These servo motors were converted to continuous servo motors by replacing the internal potentiometer that evaluates current positions with a voltage divider. Six TowerPro MG996R motors were used to actuate the fingers and wrist.

III. SOFTWARE PACKAGES USED

A. OpenCV

OpenCV is an Open-Source Computer Vision library that provides real time image processing. OpenCV is open source and cross-platform. OpenCV can be configured to run the application on a computer's graphics processing unit (GPU) or a central processing unit (CPU). Within this project, all image processing is done on the CPU. Even though GPUs are specialized for image processing, we ran the image processing on the CPU rather than the GPU to understand the minimum processing power needed for real time gesture recognition for control systems. Also, state of the art GPUs would increase the cost of the system (e.g., a NVIDIA's GPU can retail for about 1500 [16].

B. *QT Creator*: Used for code writing, in the language of C++.

C. *3D Builder*: Used for making custom 3D printable adaptations to the robot arm.

D. *Cura*: Used to prepare files for 3D printing.

IV. HOST APPLICATION

The design and implementation of the system is set to permit real time control of the robotic arm. To achieve real time results on a low-cost system the computation effort needs to be kept to a minimum. This is accomplished through filtering, isolation, and control.

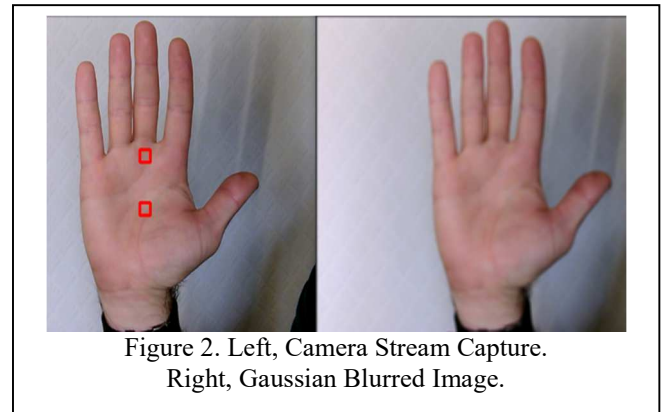
A. Filtering

Upon start-up of the gesture recognition application, the user must calibrate the system for their hand color. This is a simple method of reducing background noise and unwanted objects from the camera stream. To calibrate, the user places their hand so it completely covers the two red squares that are approximately in the center of the frame and then presses the "C" key on the keyboard. The following steps are performed to achieve hand isolation:

- A Gaussian blur is applied to the camera stream
- Spectrum conversion from Blue, Green, and Red (BGR) to Hue, Saturation, and Value (HSV)
- The HSV pixel data from the two red squares are averaged to create a lower and upper limit
- Threshold Detection
- Morphological Opening and Closing

1) Gaussian Blur

Blurring the image helps to reduce white noise and smooths edges of the image. This also removes large variances in the captured image's color spectrum which helps with threshold detection.



2) Spectrum Conversion

Converting from BGR to HSV is common practice when dealing with CV projects. Utilizing HSV enables the separation of color and intensity which helps reduces the effects of lighting issues[17,18].

3) Averaging

Hand isolation is accomplished by sampling the pixel data in the two red squares. The mean and the standard deviation of those samples are used to set the upper and lower limits for threshold detection.

4) Threshold detection

OpenCV's "inRange" function takes the upper and lower limits and filters the image. This creates a black and white image that has clearly defined edges that can then be used for the isolation process [19].

5) Morphological Opening and Closing

This process further cleans the image by eroding and dilating the hash edges left over from threshold detection. Making the contouring stage faster[20,21].



Figure 3. Left, Live camera feed.
Center, HSV converted image.
Right, Applied threshold detection.

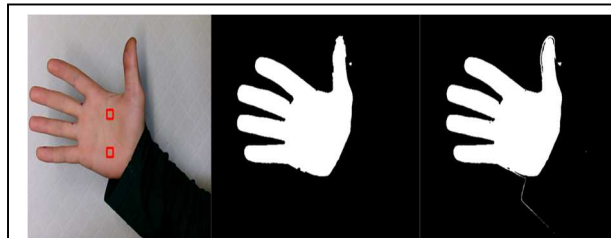


Figure 4. Left, Live camera feed.
Center, Eroded and Dilated image.
Right, Applied threshold detection.
In this image notice the sleeve on the hand being detected and the harsh thumb outline on the right image. With erosion and dilation, the size of the hand is not lost while unwanted data is filtered.

B. Isolation

Identifying key features of the hand is paramount for gesture recognition. Finding these features is difficult. The following steps are taken to accomplish real time tracking and hand gesture recognition:

- Edge detection and evaluation
- Finger detection
- Center of palm evaluation

1) Edge Detection and Evaluation

Applying edge detection becomes a simple manageable process of using OpenCV's "findContours" function. This adds all the continuous points along a boundary to a vector which can be draw and evaluated. If the image has more than one boundary, sorting is required to find the contour that represents the hand. This should be the contour with the largest area. Sometimes smaller boundaries are detected, and this additional step ensures the hand is located quickly and accurately.

2) Finger Detection

Creating a convex hull around the contour of the hand supports finding the fingertips and interdigital folds. The process looks for intersecting points between the hull and the contour. These points are classed as fingertips. The furthest

points from the hull along the contour are classed as interdigital folds.

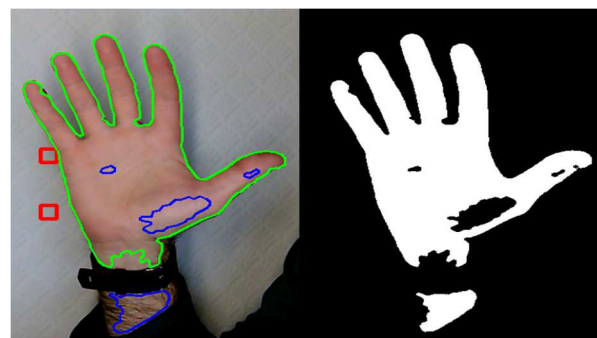


Figure 5. Left, the largest contour is drawn in green and smaller contours are drawn in blue.
Right, Applied threshold detection with multiple boundaries.

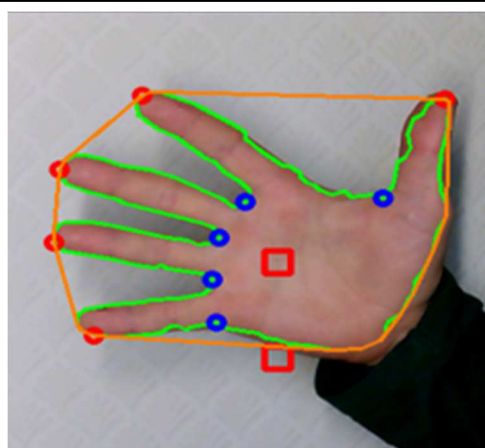


Figure 6. Orange Line is the convex hull.
Green line is the hand contour.
Red dots are the fingertips.
Blue dots are the interdigital folds.

3) Center of Palm

Evaluating for the center of the palm takes careful consideration. The center of the palm needs to be a stable point and easy to find. It must be as accurate as possible because the center of the palm is used to evaluate the extension of the fingers as well as to control the arm's movements. Three methods of evaluating the palms center were tested. The first is the actual geometric center, the centroid. This provides the accurate location of the center of the palm. However, this becomes a problem each time the contour changes shape, due to lighting or shadows, losing much needed precision for the control system. The second method was to find the center of a minimum inclosing circle around the hand. This method was very precise, but its accuracy was slightly off. The third method is like the second, but it uses a minimum enclosing rectangle. In figure 7, all the center locating methods are visually close. In a recorded test, each center locating method was plotted for 20 seconds, with an unmoving hand. It was observed that the standard deviation of the precision was significantly less for the minimum enclosing rectangle than the other methods, while also maintaining acceptable accuracy.

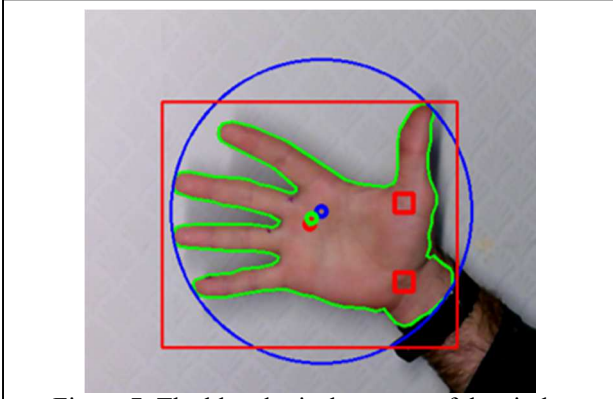


Figure 7. The blue dot is the center of the circle.
The red dot is the center of the rectangle.
The green dot is the centroid of the hand.

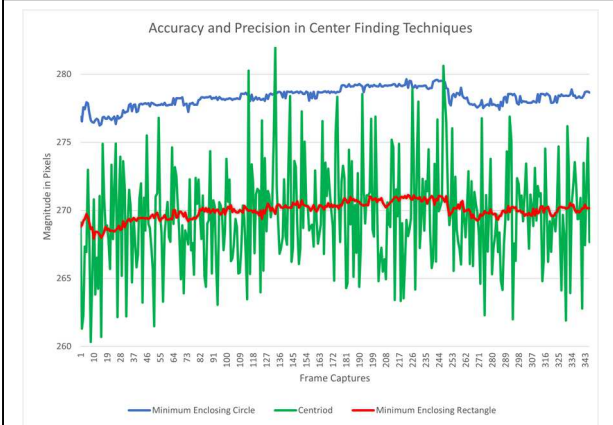


Figure 8. Blue is minimum enclosing circle center.
Red is minimum enclosing rectangle center.
Green is the geometric centroid.

C. Control

Evaluating the position of the fingertips, defects, and center of the palm is what supports the constructions of a robust yet simple control system. This is broken down into two main parts: quadrant control and hand pose detection. The current control system uses a quadrant-based approach to enable or disable arm movements. This creates a safe environment for the robot arm to operate in.

1) Quadrant control

Quadrant control is evaluated using a derivation of the cross-product formula (1) which tests to see where the hand lies.

$$\begin{aligned} \text{Point evaluated} &= (x, y) \\ \text{Vector evaluated against} &= (x_1, y_1), (x_2, y_2) \\ D &= (x - x_1)(y_2 - y_1) - (y - y_1)(x_2 - x_1) \end{aligned} \quad (1)$$

If the value of D is positive, the point resides on one side of the line. If the value of D is negative, the point is on the opposite side of the line. If the value is equal to zero, the point is on the line. For this control system, if the point is evaluated to being on the line it is treated as being across the line. The point that is evaluated is the center of hand. There are 16 different regions that center of the hand may reside in but only nine different commands issued as seen in figure 9.

30% of window frame Width		50% of window frame Width	70% of window frame Width	
Shoulder Left Bicep Extend Hand Pose Detection Disabled	Shoulder Bicep Hand Pose Detection Disabled	Disabled Bicep Extend Disabled	Shoulder Right Bicep Extend Hand Pose Detection Disabled	30% of window frame Height
Shoulder Left Bicep Disabled Hand Pose Detection Disabled	Shoulder Bicep Hand Pose Detection Enabled	Disabled Bicep Disabled	Shoulder Right Bicep Disabled Hand Pose Detection Disabled	50% of window frame Height
Shoulder Left Bicep Curl Hand Pose Detection Disabled	Shoulder Bicep Hand Pose Detection Disabled	Disabled Bicep Curl Disabled	Shoulder Right Bicep Curl Hand Pose Detection Disabled	70% of window frame Height

Figure 9. Regions and their controls

To minimize computation effort while tracking the hand position, four tests are performed. The outcome of these tests is either a “1” or a “0”. Using Boolean logic, the four test outputs are combined into a 4-bit number. Figure 10 shows the truth table for the combined logic. The first test determines if the center of the palm is above the X-axis, or 50% of the window frame height. The second test determines if the center of the palm is to the right of the Y-axis or 50% of the window frame width. Depending on the answers to these first two questions, the secondary set of questions change. For example, if the answer to question 1 is “yes”, then the X-boundary is set to 30% of the window frame height. If the answer to question 1 is “no”, the X-boundary is set to 70% of the window frame height, as seen in Figure 9. The same logic is used for determining the Y location. Once all four tests are completed, the 4-bit number is used in a Look-Up-Table (LUT) to enable or disable arm movements.

Combinational Logic	Not Right of Y boundary	Not Right of Y - axis	Right of Y-axis	Right of Y boundary
Above X boundary	1010	1011	1110	1111
Above X - axis	1000	1001	1100	1101
Not above X - axis	0010	0011	0110	0111
Not Above X boundary	0000	0001	0100	0101

Figure 10. Bit ordering from left(MSB) to right(LSB)

2) Hand Pose Detect

When detecting a hand pose, the application compares the average value of the fingertips to the average value of the defects, relative to the center of the palm. A cross cultural study of hand dimensions shows that approximately from the center of the palm to the fingertips is 1.86 times longer than from the center of the palm to the defects [22]. Applying this ratio allows for more accurate detection between hand poses at variable distances from the camera. In Figure 11, accurate detection ranges from 0.3 – 1.0 meters distance from the camera.

D. Communication

This project use internet protocols to establish communication from the host computer to the Raspberry Pi over a wireless local area network (WLAN). The host computer transmits a command word that details what servo motors need to move and by how much. On the Raspberry Pi,

when the command word is received, it decodes each section of the word and issues the command to each servo motor.

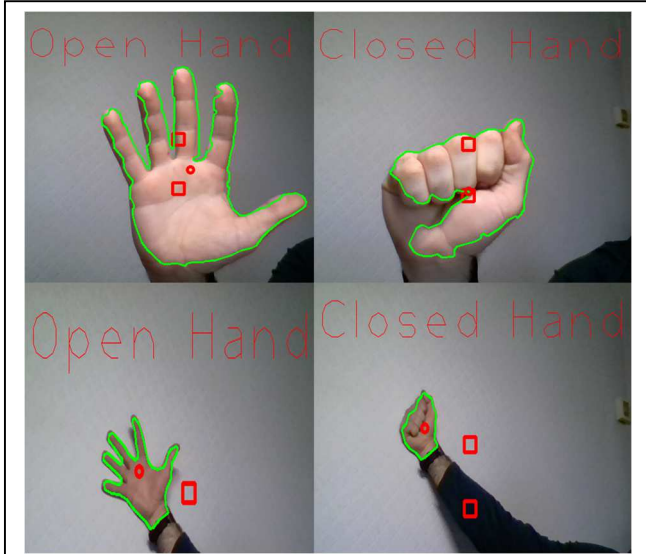


Figure 11. Top left and Top Right show Open and Closed hand detection at 0.3 meters from the camera. Bottom Left and Bottom Right show Open and closed hand detection at 1.0 meters from the camera.

E. System Flow Charts

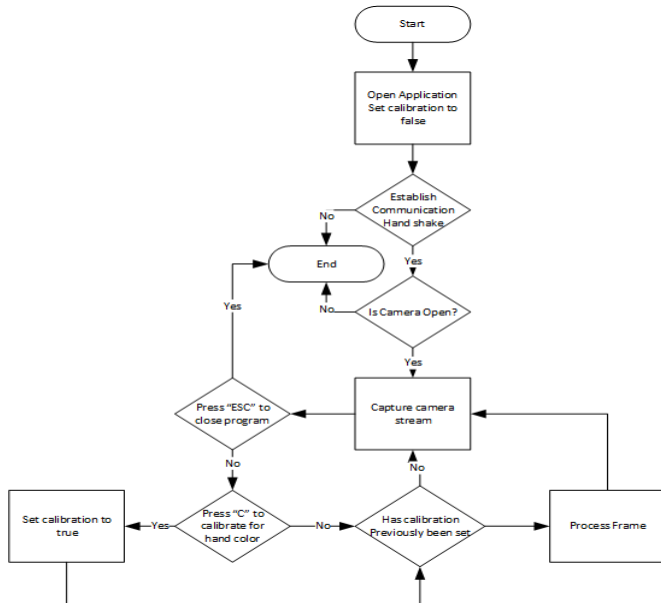


Figure 12. Flow chat of the overall system

V. THE ROBOTIC ARM

The robotic arm is a simple 3D printed model of the human arm that has 360-degree rotation in the shoulder, 90-degree extension in the bicep, 180-degree rotation in the wrist, and fingers that can curl and extend. Each of these movements require a servo motor and PWM control from the Raspberry Pi. An interrupt system monitors push buttons that act as simple pressure sensors to ensure the arm operates within its load limitations.

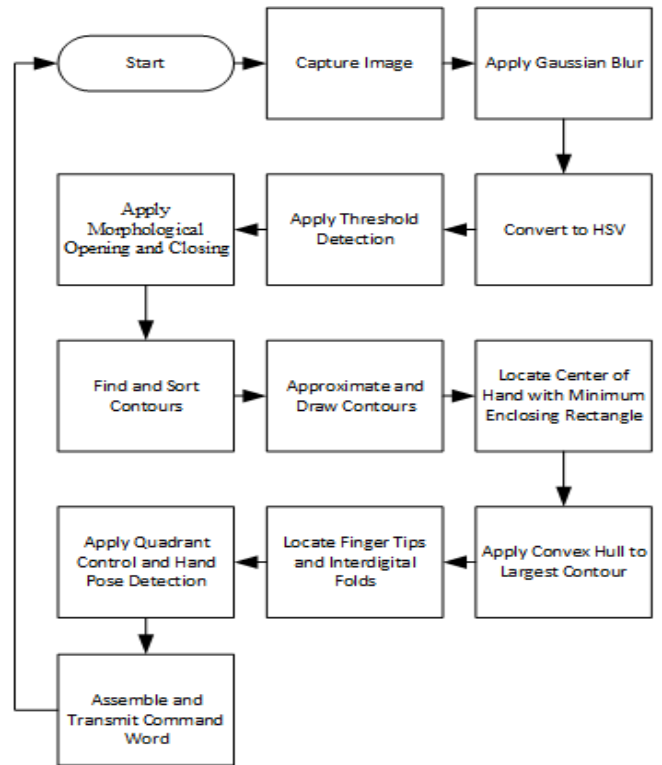


Figure 13. Flow chart of the steps that are occurring inside of the "Process Frame" of Figure 12's system flow chart.

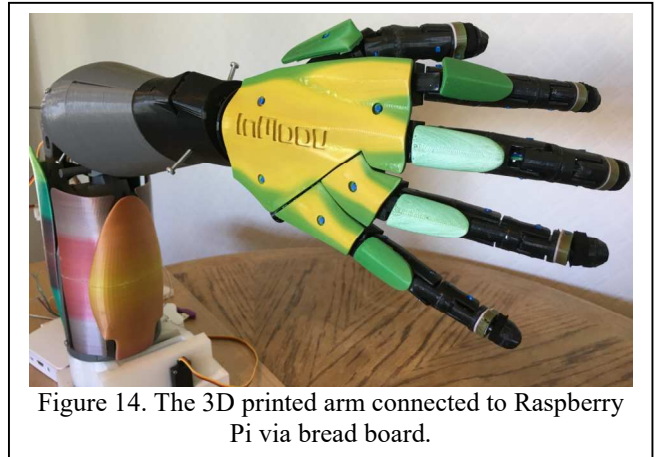


Figure 14. The 3D printed arm connected to Raspberry Pi via bread board.

VI. RESULTS

Testing the control system, the robotic arm responded in real time to both directional and hand pose detection commands. The recognizing gestures of the hand proved successful in a testing environment. The limit switches on the robotic arm worked to stop movement when they were pressed preventing damage to items being held or the robotic arm, but there is room for improvement.

A. Suggested Improvements

The first recommendation is to explore a better method of calibrating for hand color or apply a form of background subtraction that is computationally effortless.

The second recommendation is to apply a detection method for individual finder movement and articulation. This could be accomplished by splitting the hand contour into smaller vectors and evaluation their position or magnitude relative to other segments.

The third recommendation is to have the Raspberry Pi communicate with a smaller microcontroller such as the Arduino Mega. This would require a voltage level shifter to communicate via I²C protocol or communicate with a wireless module such as an nRF24L01 module. There are three reasons to choose an Arduino over Raspberry Pi.

1) PWM Control

With the Raspberry Pi, PWM control is taxing on the CPU. With each additional PWM wave generated, the timing accuracy of the Raspberry Pi diminishes. The Raspberry Pi only has one natural PWM control output port and the rest must be generated with the support of software writing library for PWM signals. The Arduino Mega has 15 PWM ports and could easily control the servo motors.

2) Analogue Inputs

The Raspberry Pi does not contain any analogue input pins. This requires an analogue to digital converter to process signals from pressure sensors. This is important because more precise pressure sensors could be used for delicate handling of material. The Arduino Mega has 16 analogue input pins making it perfect for numerous sensors to be placed on the robotic arm.

3) Library Support

The Arduino Integrated Development Environment contains servo motor specific functions that could prove beneficial to this or similar projects. The “servo read” function can provide current position of the servo motor which would benefit an interrupt system based on pressure.

VII. CONCLUSION

Complex computer vision-based gesture recognition systems are being developed that are capable of interpreting sign language in real time. Those system will be interfacing with neural networks such as Google’s TensorFlow and system requirements on computer processing will be much higher because the application will be configured to operate on specialized GPUs [23]. Considering, a low-cost gesture recognition application to be used as a control method for robotics, this project proves that simple control system can be built using computer vision.

VIII. ACKNOWLEDGEMENT

This publication has been financially supported by the University of Nottingham/ Electrical and Electronic Department. Thanks to Dr. Kevin Webb and Dr. Steve Bull

IX. REFERENCES

- [1] V. I. Pavlovic, R. Sharma and T. S. Huang, "Visual Interpretation of Hand Gestures for Human-Computer Interaction: A Review," in *IEEE Xplore*, 1997.
- [2] H. Zhou and H. Hu, "Humanmotion tracking for rehabilitation," *Biomedical Signal Processing and Control*, vol. 3, no. 1, pp. Pages 1-18, 2008.
- [3] J. J. LaViola Jr., "A survey of Hand Posture and Gesture Recognition Techniques and Technology," NSF Science and Technology Center for Computer Graphics and Scientific Visualization, Providence, 1999.
- [4] M. Zhou, X. Chai, G. Li, Y. Ln, Z. Xu, Y. Tang and X. Chen, "Sign Language Recognition and Translation with Kinect," Beijing, 2013.
- [5] J. Oliensis, D. Samaras, S. Lu and D. Metaxas, "Using Multiple Cues for Hand Tracking and Model Refinement," in *Computer Society Conference on Computer Vision and Pattern Recognition*, Piscataway, 2003.
- [6] Virtual Realities, LLC, "Virtual Realities, LLC," Virtual Realities, LLC, 2018. [Online]. Available: <http://www.vrealities.com/products/data-gloves>. [Accessed 12 03 2021].
- [7] D. Prattichizzo, M. Mohammadi, L. Meli, S. Scheggi and T. L. Baldi, "GESTO: a Glove for Enhanced Sensing and Touching Based on Inertial and Magnetic Sensors for Hand Tracking and Cutaneous Feedback," in *Transactions on Human-Machine Systems*, Siena, 2017.
- [8] PATSEER PRO, "Hand Gesture Recognition," Gridlogics, Baner, 2017.
- [9] OpenCV, "Open Source Computer Vision," doxygen, 18 7 2020. [Online]. Available: https://docs.opencv.org/3.4/d1/dc5/tutorial_background_subtraction.html. [Accessed 12 3 2021].
- [10] DesignSpark, "RS Componets," DesignSpark, [Online]. Available: [https://uk.rs-online.com/web/p/raspberry-pi-cameras/1843079/?cm_mmc=UK-PLA-DS3A--google--CSS_UK_EN_Raspberry_Pi_%26_Arduino_%26_Development_Tools_Whoop--Raspberry+Pi+Cameras_Whoop+\(2\)-_-1843079&matchtype=&pla-300343911333&gclid=Cj0KCQiAv6yCBhCLARIsABqJ](https://uk.rs-online.com/web/p/raspberry-pi-cameras/1843079/?cm_mmc=UK-PLA-DS3A--google--CSS_UK_EN_Raspberry_Pi_%26_Arduino_%26_Development_Tools_Whoop--Raspberry+Pi+Cameras_Whoop+(2)-_-1843079&matchtype=&pla-300343911333&gclid=Cj0KCQiAv6yCBhCLARIsABqJ). [Accessed 12 3 2021].
- [11] Sony, "Sony," Sony, 11 9 2015. [Online]. Available: https://pro.sony/en_GB/products/handheld-camcorders/pxw-fs5#PROR301008ProductsSpecificationsblock-pxw-fs5. [Accessed 12 03 2021].
- [12] B. Michaelis, J. Appenrodt, A. Al-Hamadi and M. Elmezain, "A Hidden Markov Model-Based Continuous Gesture Recognition System for Hand Motion Trajectory," in *2008 19th International Conference on Pattern Recognition*, Germany, 2008.
- [13] ASUS, "Notbook PC E-Manual," ASUSTek COMPUTER INC., Peitou, 2013.
- [14] HUE, "HUE," HUE, 2021. [Online]. Available: <https://huehd.com/shop/hue-hd-camera/?ph=782f5d1b5f6507f8e0765137>. [Accessed 15 03 2021].
- [15] G. Langevin, "InMoov," InMoov, [Online]. Available: <http://inmoov.fr/>. [Accessed 15 3 2021].
- [16] NVIDIA, "NVIDIA," NVIDIA, 2021. [Online]. Available: <https://shop.nvidia.com/en-gb/geforce/store/gpu/?page=1&limit=9&locale=en-gb&category=GPU&gpu=RTX%203090>. [Accessed 15 03 2021].
- [17] V. Rajini and G. P, "Assessment of satellite image segmentation in RGB and HSV color space using image quality measures," in *2014 International Conference on Advances in Electrical Engineering*, Vellore, 2014.
- [18] S. Pramanik, G. Qian and S. Sural, "Segmentation and histogram generation using the HSV color space for image retrieval," in *International Conference on Image Processing*, Rochester, 2002.
- [19] R. Shilkrot, V. G. Mendonca, P. Joshi and D. M. Escriva, Building Computer Vision Projects with OpenCV 4 and C++, Brimingham: Packt Publishing, 2019.
- [20] J. Xiaojun, "Fabric defect detection based on open source computer vision library OpenCV," in *International Conference on Signal Processing Systems*, Dalian, 2010.
- [21] W. Lu and G. Xie, "Image Edge Detection Based On Opencv," *International Journal of Electronics and Electrical Engineering*, vol. 1, no. 2, pp. 104-106, 2013.
- [22] O. Yoldas, O. Cetik, B. Durgun and E. Cakit, "A Survey of Hand Anthropometry and Biomechanical Measurements of Dentistry Students in Turkey," *Human Factors and Ergonomics in Manufacturing & Service Industries*, vol. 24, no. 6, pp. 739 - 753, 2018.
- [23] J. Zhang, Q. Gong and Z. Zeng, "CNN Model Design of Gesture Recognition Based on Tensorflow Framework," in *Information Technology, Networking, Electronic and Automation Control Conference*, Chengdu, 2019.